Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Дисциплина
«Информационная безопасность»

Лабораторная работа №7

Безопасность браузера и анализ сетевого трафика

Выполнил: Малышев Никита Александрович
Группа: P3412
Проверила: Маркина Татьяна Анатольевна

Санкт-Петербург
2026 год

Содержание

1. Настройка браузера 3
2. Анализ трафика 17
3. Как чаще всего передаются данные аутентификации (логин/пароль) в современных веб-приложениях? 27
4. Вывод 27

1. Настройка браузера

Для выполнения данной работы был выбран браузер Google Chrome. Первым этапом стало изучение параметров конфиденциальности и безопасности (Privacy and security), доступных в настройках браузера.

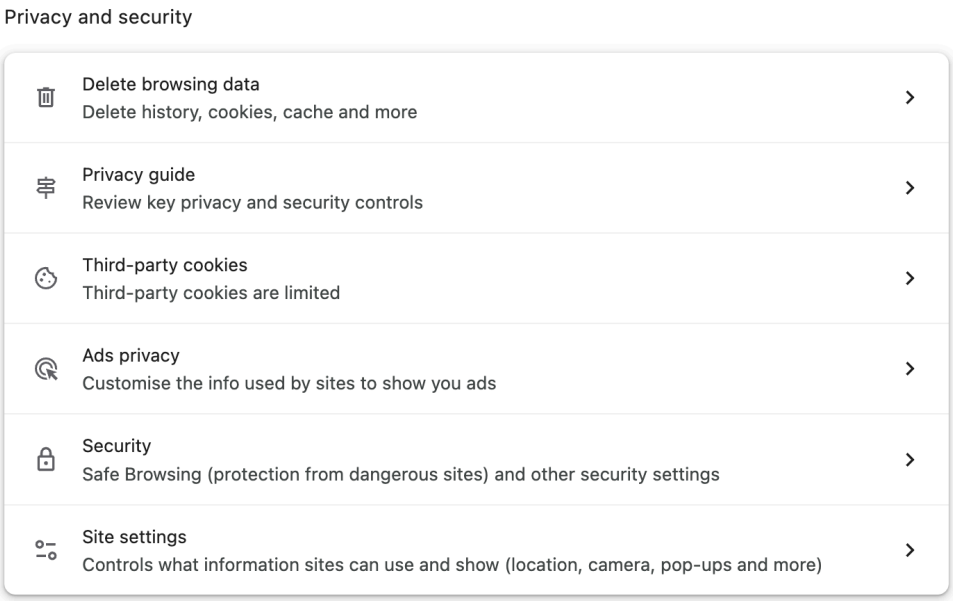


Рис. 1. Настройки конфиденциальности и безопасности

Откроем раздел Third-party cookies (Сторонние файлы cookie)

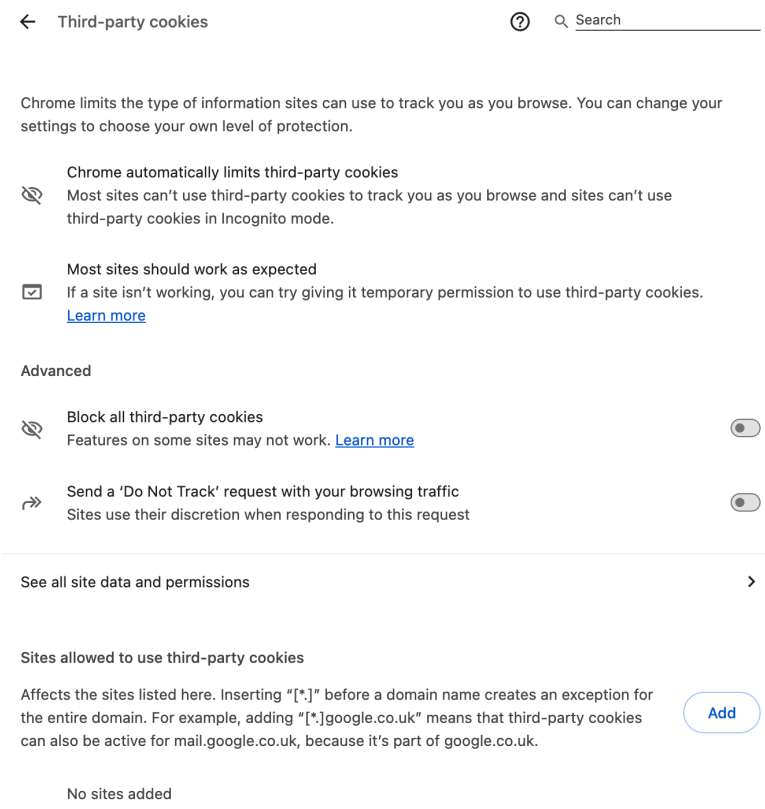


Рис. 2. Начальные настройки сторонних файлов cookie

Исходная конфигурация браузера предполагала разрешение на использование сторонних cookie-файлов. Данная настройка позволяла доменам, отличным от текущего посещаемого сайта, сохранять и использовать свои файлы cookie. Такая практика широко применяется в рекламных сетях для отслеживания пользовательской активности на различных интернет-ресурсах, включая социальные сети, с целью формирования поведенческого профиля и демонстрации таргетированной рекламы. В ходе настройки указанная функция была принудительно отключена путём активации опции «Block all third-party cookies» (Блокировать все сторонние cookie).

Также была активирована опция «Send a „Do Not Track“ request» (Отправлять запрос «Не отслеживать»). Данный HTTP-заголовок информирует веб-серверы о предпочтении пользователя не быть отслеживаемым. Важно отметить, что использование этого заголовка носит рекомендательный характер, и веб-сайты не обязаны его соблюдать, однако в сочетании с жёсткой блокировкой cookie он повышает общий уровень конфиденциальности.

В рамках политики блокировки был рассмотрен белый список исключений (Sites allowed to use third-party cookies). Этот список позволяет предоставить доступ к сторонним cookie для конкретных доверенных доменов, даже при общем запрете. В данном случае список был оставлен пустым, без добавления каких-либо исключений, что обеспечивает максимально строгий режим блокировки для всех сайтов без исключений.

Далее настройки были продолжены в разделе, посвященном конфиденциальности в рекламе (Ads privacy).

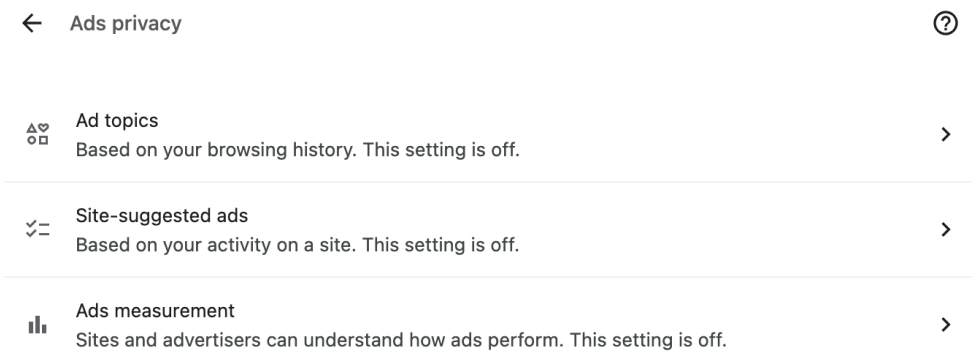


Рис. 3. Раздел Ads privacy

Откроем подраздел Ad topics (Темы рекламы)

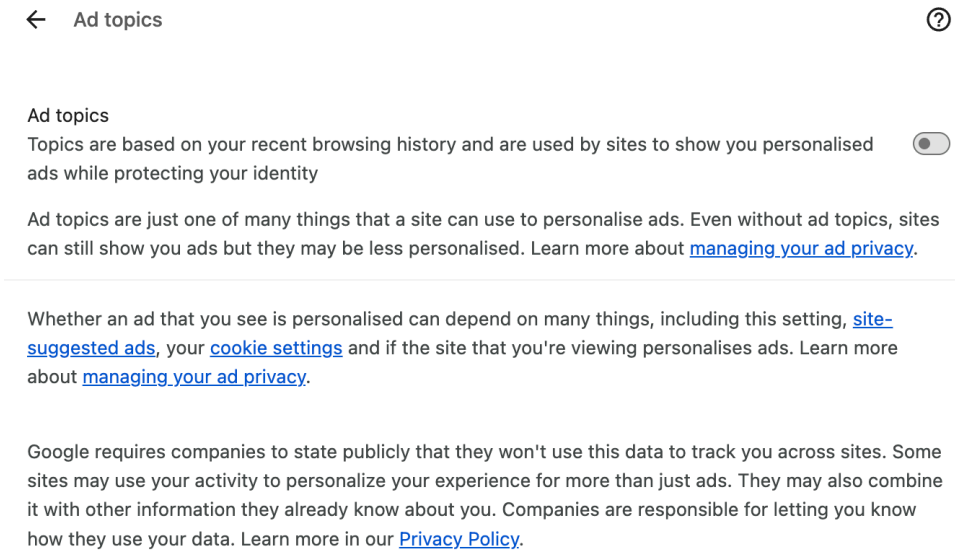


Рис. 4. Подраздел Ad topics

В браузере Chrome реализована альтернативная технология, призванная заменить традиционные сторонние cookie. Её принцип действия заключается в анализе истории просмотров пользователя с целью его автоматического отнесения к определённым тематическим группам, таким как «автомобилисты» или «кулинары». Вместо передачи индивидуального идентификатора сайты получают информацию о принадлежности к такой обобщённой группе. Данная функция была отключена, поскольку, несмотря на декларируемую анонимность, её суть остаётся в отслеживании поведения пользователя для показа релевантной рекламы.

Далее настройки были продолжены в подразделе Site-suggested ads (Предлагаемая сайтами реклама).

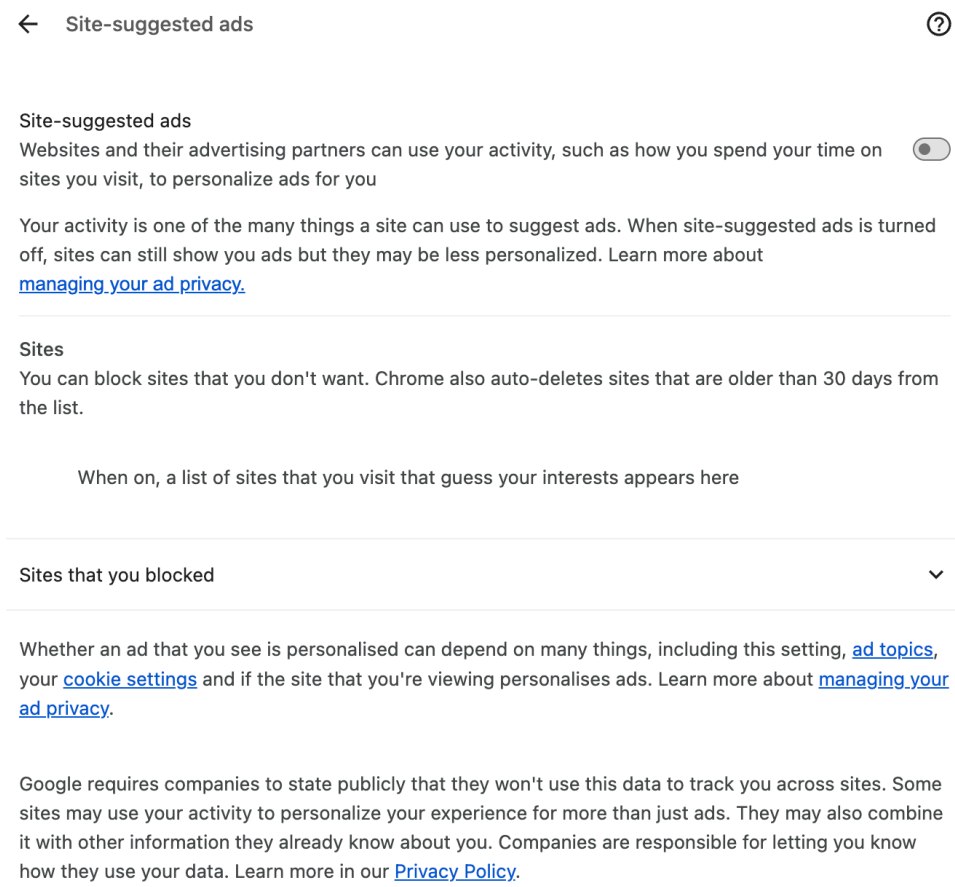


Рис. 5. Подраздел Site-suggested ads

Отдельный веб-сайт обладает возможностью использовать информацию о пользовательской активности в его пределах — просмотренные страницы, продолжительность сеансов и другие взаимодействия — для демонстрации релевантной рекламы как на своей собственной платформе, так и через сеть партнерских ресурсов. Это позволяет расширить таргетирование за пределы одного домена. Выключаем эту функцию.

Перейдем в подраздел Ads measurement (Оценка эффективности рекламы).

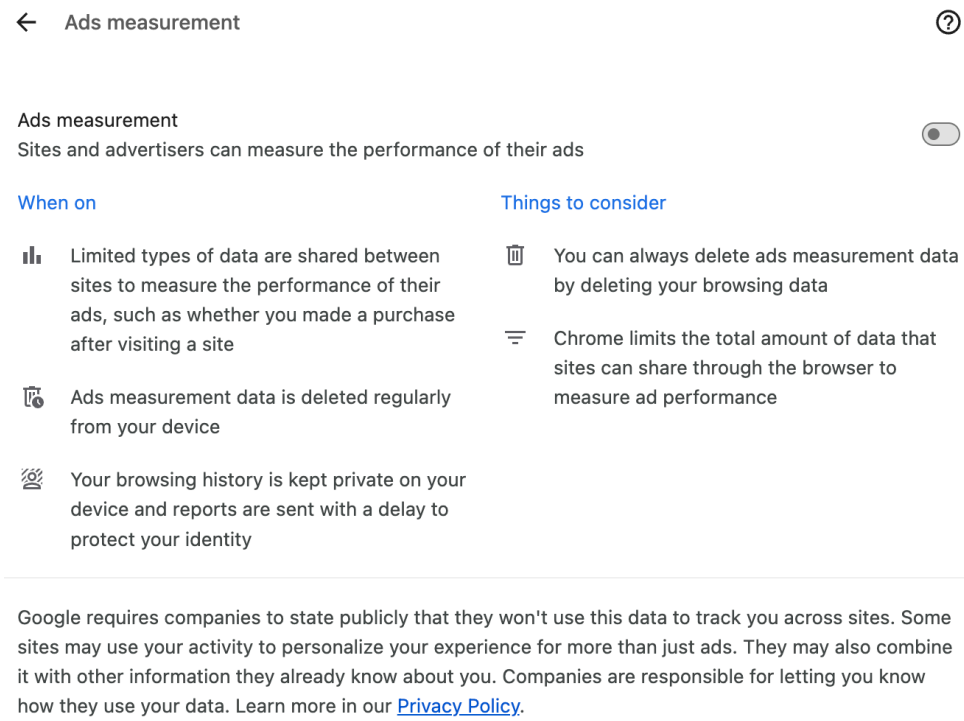


Рис. 6. Подраздел Ads measurement

Данная функция предоставляет рекламодателям аналитику о результативности их кампаний, позволяя оценить, привели ли показы объявлений к целевым действиям, таким как совершение покупки после клика. Для формирования этих отчетов с устройства пользователя могут передаваться обезличенные сведения о конверсиях. Эта функция была отключена, поскольку, несмотря на заявления о применении анонимизации и ограниченном характере собираемых данных, её работа по своей сути предполагает передачу информации о поведении пользователя третьим сторонам — рекламодателям.

Откроем раздел Security (Безопасность).

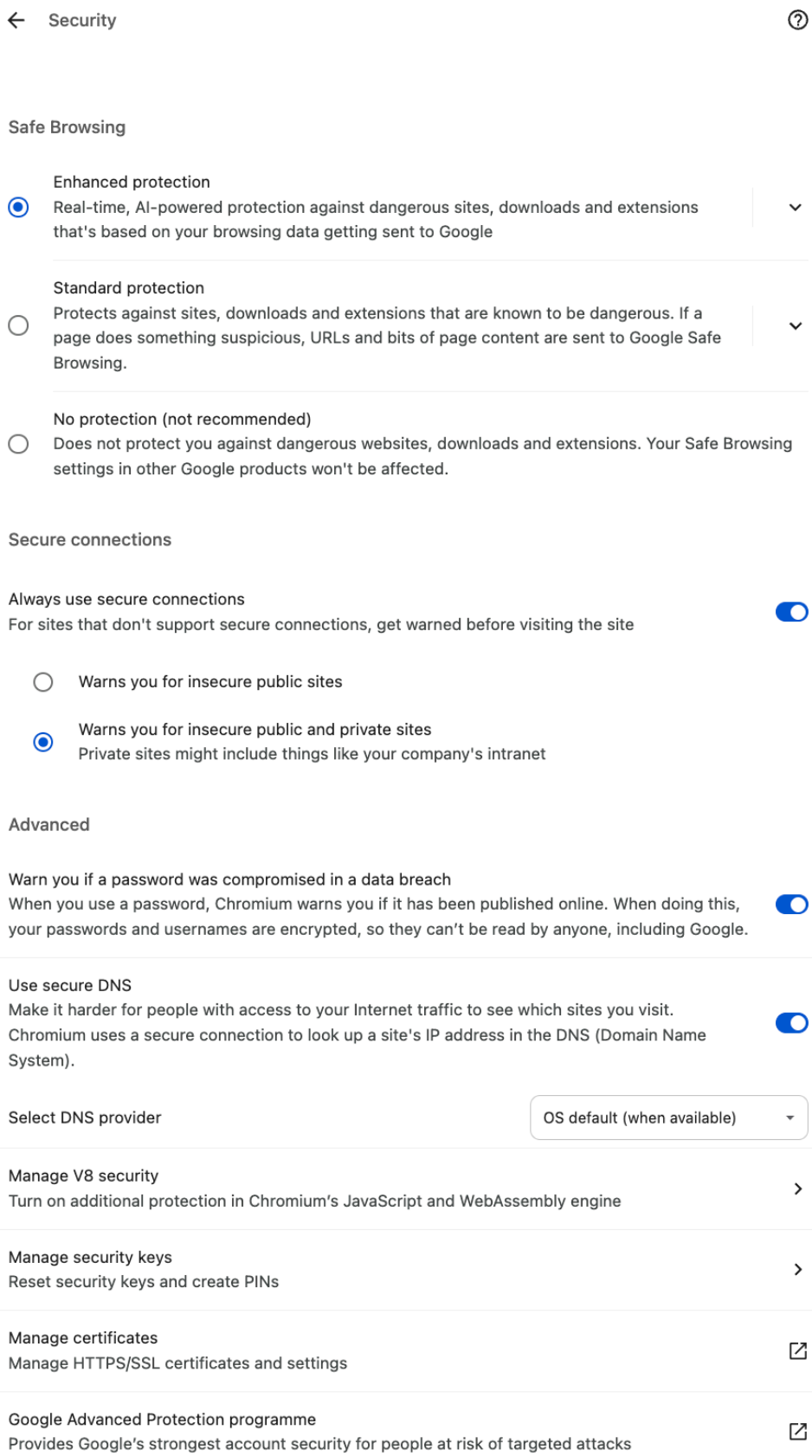


Рис. 7. Раздел Security

Было решено включить усиленный уровень защитных механизмов (Enhanced Protection), поскольку он представляет собой наиболее надеж-

ное решение, обеспечивающее максимальную безопасность при работе в сети. Enhanced Protection — это режим безопасности Chrome, который в реальном времени проверяет посещаемые страницы и загрузки по постоянно обновляющимся базам Google Safe Browsing, а также анализирует подозрительное поведение сайтов. Данный режим не только активно противодействует фишинговым атакам и блокирует доступ к вредоносным ресурсам, но и заранее предупреждает о потенциально опасных сайтах и загрузках, а также блокирует известные трекеры. Это делает использование браузера существенно более безопасным и конфиденциальным по сравнению со стандартным уровнем защиты.

Оставляем функцию сообщения о раскрытии паролей, тогда можно будет быстрее поменять пароль в случае утечки и снизить риски получения несанкционированного доступа к аккаунту. Google все равно не видит пароли в чистом виде, а проверка происходит на основе хешей паролей.

Также была активирована опция использования защищенного DNS-сервера, что повышает степень конфиденциальности. При ее использовании DNS-запросы шифруются, что ограничивает возможность интернет-провайдера отслеживать историю посещаемых пользователем сайтов. Оставляем функцию Always use secure connections (Всегда использовать безопасные соединения), при попытке перехода на любой сайт с устаревшим протоколом HTTP браузер будет блокировать загрузку и выводить предупреждение.

Перейдем в подраздел V8 optimiser (Управление безопасностью V8).

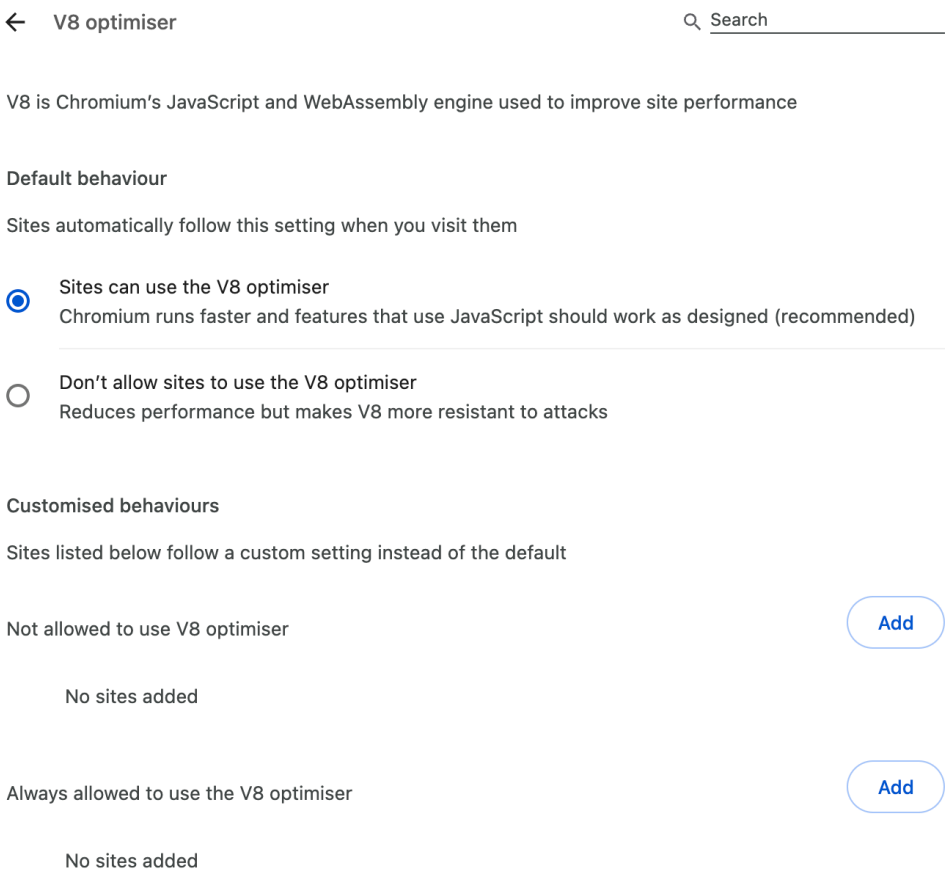


Рис. 8. Подраздел V8 optimiser

Настройки, связанные с оптимизацией выполнения JavaScript, были оставлены в конфигурации по умолчанию, так как применяемые браузером механизмы оптимизации критически важны для обеспечения стабильной работы современных веб-приложений, что соответствует потребностям подавляющего большинства пользователей.

Перейдем в подраздел Local certificates (Локальные сертификаты).

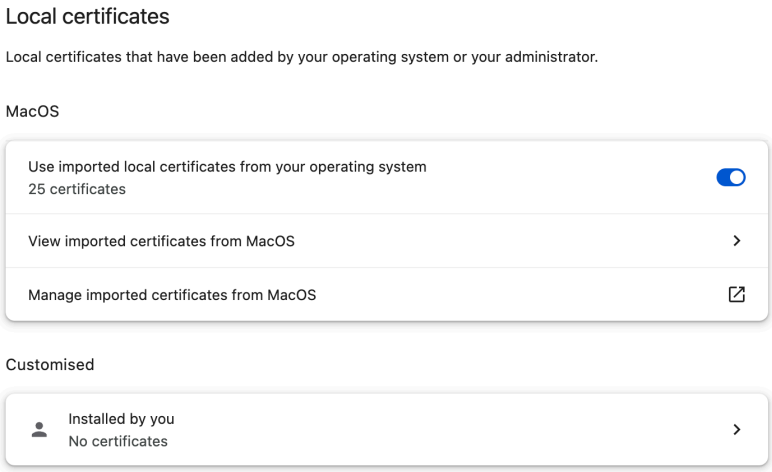


Рис. 9. Подраздел Local certificates

В интерфейсе управления сертификатами наблюдается, что браузер интегрирован с хранилищем сертификатов операционной системы, используя 25 доверенных корневых сертификатов. Эта интеграция является фундаментальной для обеспечения безопасного и корректного установления соединений с веб-ресурсами, использующими протокол HTTPS.

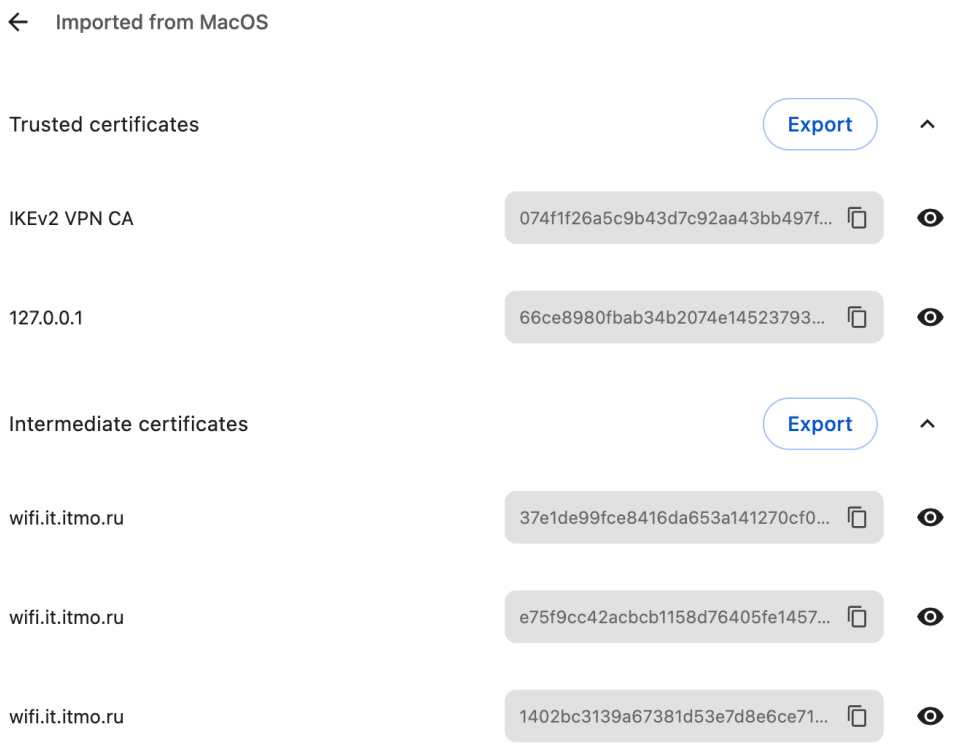


Рис. 10. Импортированные из MacOS сертификаты

Перейдем в раздел Site settings (Настройки сайтов).

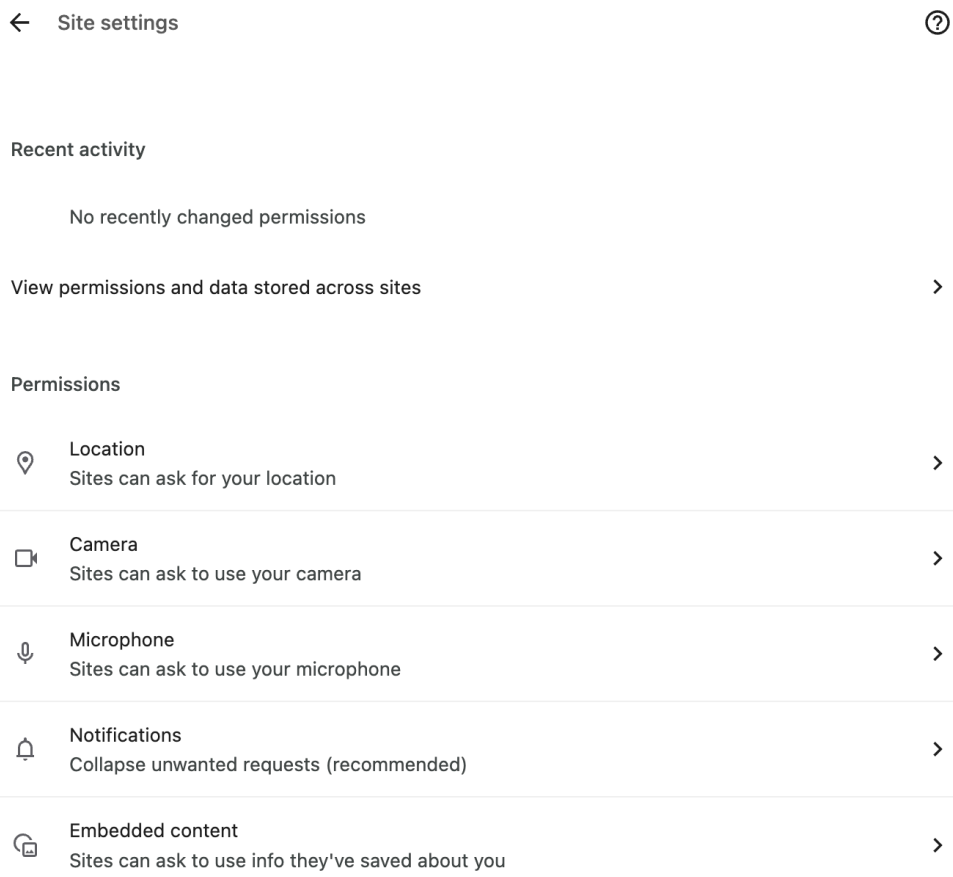


Рис. 11. Основные разрешения сайтов

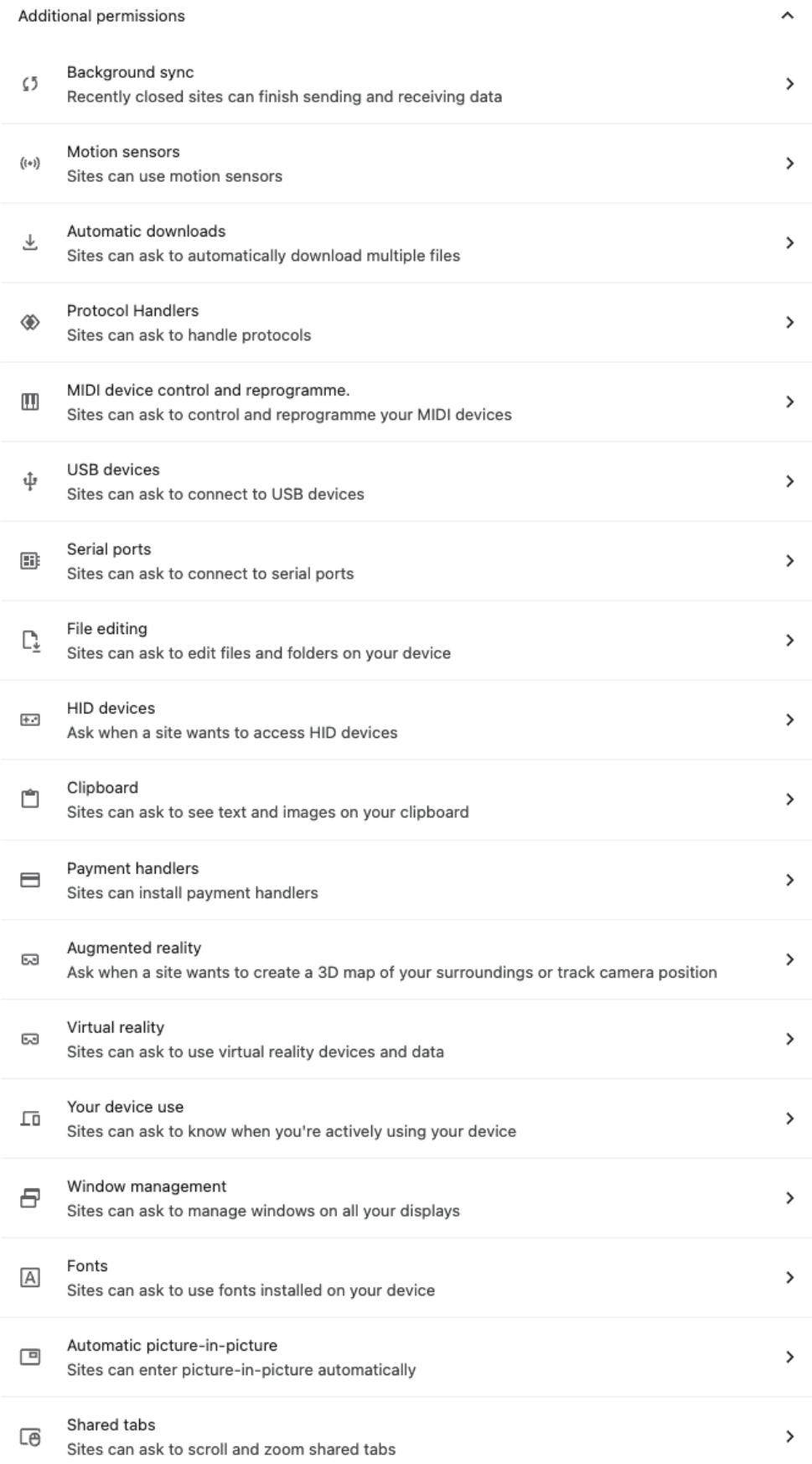


Рис. 12. Дополнительные разрешения сайтов

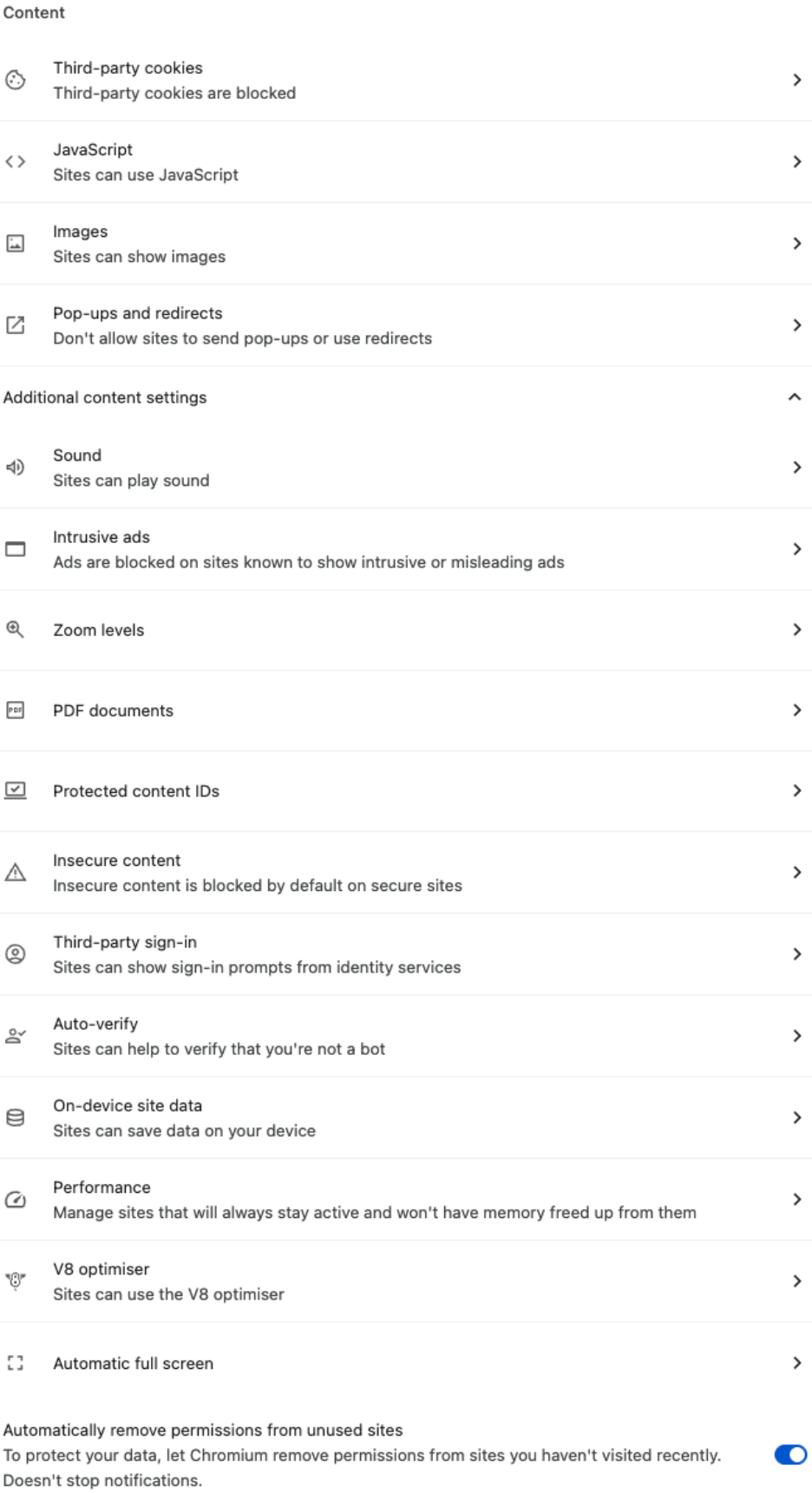


Рис. 13. Контент и дополнительные настройки контента

Проведем политику максимального ограничения доступа: Отключены опасные функции доступа к USB-устройствам, файловой системе, портам, датчикам, midi-устройствам. Запрещена фоновая синхронизация и автоматические загрузки. Активировано автоматическое удаление разрешений для неиспользуемых сайтов.

Было сохранено разрешение на выполнение JavaScript-сценариев, поскольку эта функция является фундаментальной для обеспечения полной функциональности и интерактивности подавляющего большинства современных веб-сайтов. Также было разрешено отображение графических элементов, что необходимо для корректного восприятия контента.

Для доступа к таким аппаратным средствам, как микрофон и камера, а также к данным о местоположении, был установлен режим «Спрашивать разрешение». Данная настройка гарантирует, что браузер будет каждый раз запрашивать явное согласие пользователя при попытке сайта получить доступ к этим ресурсам, предоставляя возможность предоставлять доступ только в случае действительной необходимости.

Далее очистим кэш и куки.

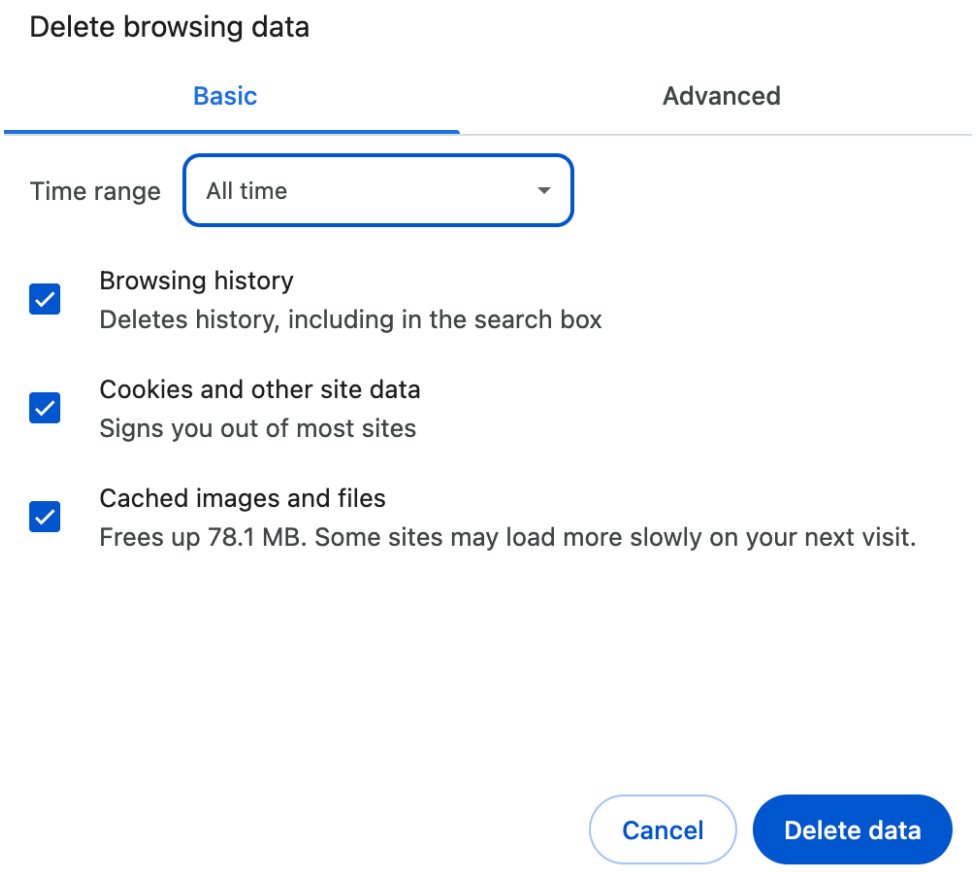


Рис. 14. Контент и дополнительные настройки контента

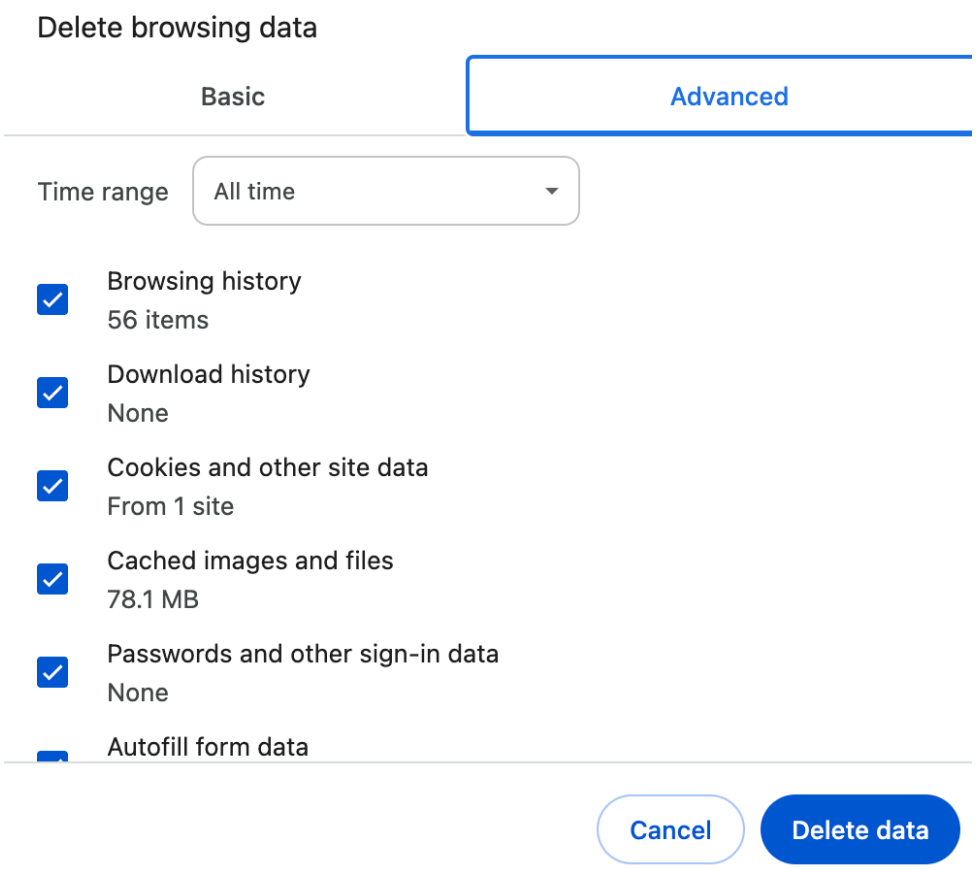


Рис. 15. Дополнительные настройки удаления данных браузера

После очистки кэша и куки освободилось более 100 МБ пространства.

Отключим функции Offer to save passwords and passkeys (Предлагать сохранение паролей и ключей доступа) и Sign in automatically (Автоматический вход) в Password Manager (Менеджер паролей) и удалим все сохранённые пароли (у меня их не было).

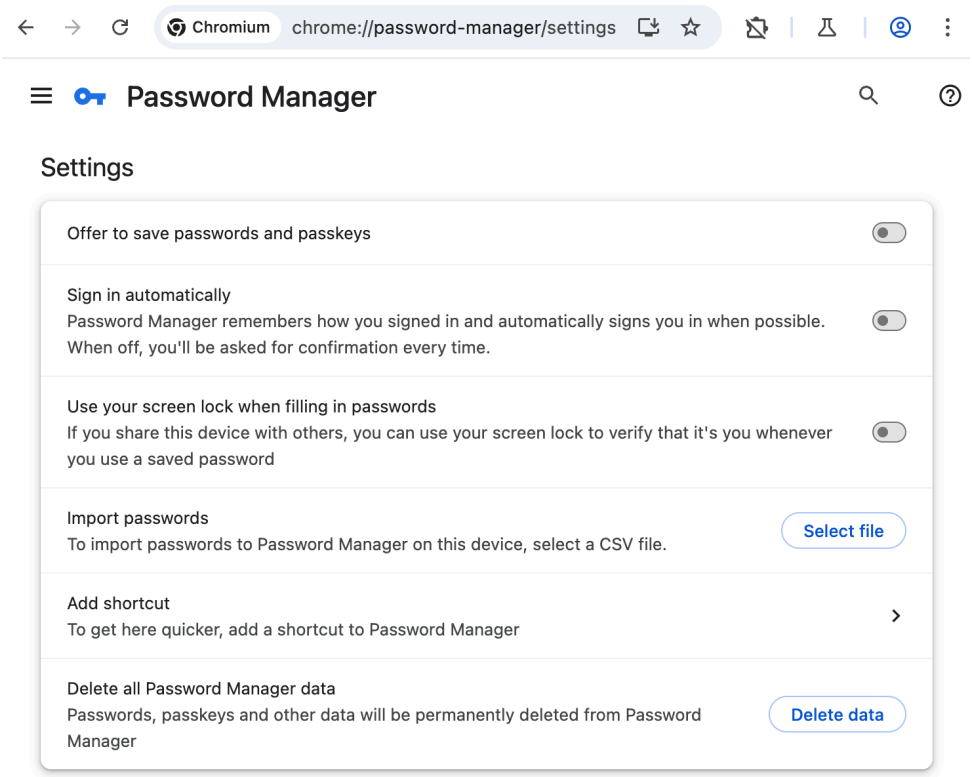


Рис. 16. Настройка менеджера паролей

2. Анализ трафика

Далее зайдем на сайт https://streamcraft.net/. Перейдем во вкладку Network (Сеть). Видим, что при обычной загрузке страницы было послано 86 HTTP-запросов. Зайдем в заголовки самого первого GET-запроса.

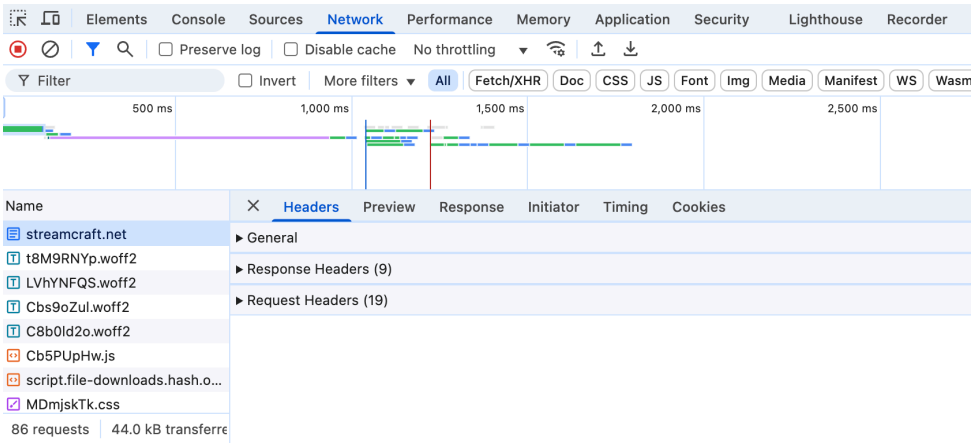


Рис. 17. Типы заголовков первого GET-запроса

Видим, что тут находится три типа заголовков — общие, заголовки ответов и запросов.

Посмотрим на общие заголовки.

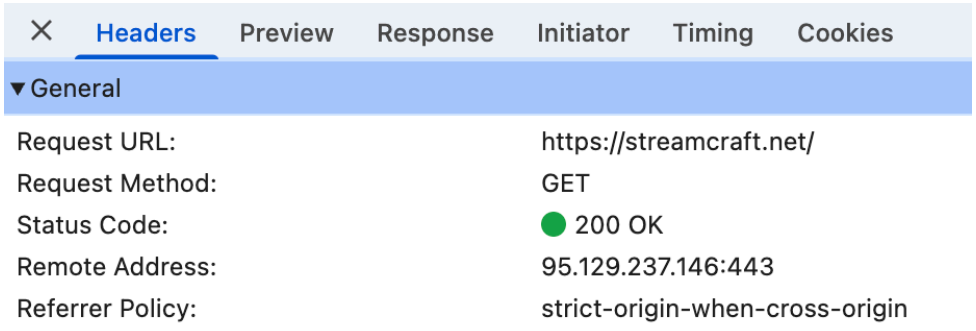


Рис. 18. Общие заголовки

Тут представлены: код статуса, удаленный адрес (IP сервера `streamcraft.net` + порт 443, стандартный для https протокола) и правило перехода, контролирующее, какую информацию отправлять при переходе между сайтами (strict – строгое ограничение, origin – передается только домен, но не полный URL, cross-origin – при переходе на другой сайт).

Рассмотрим заголовки ответа.

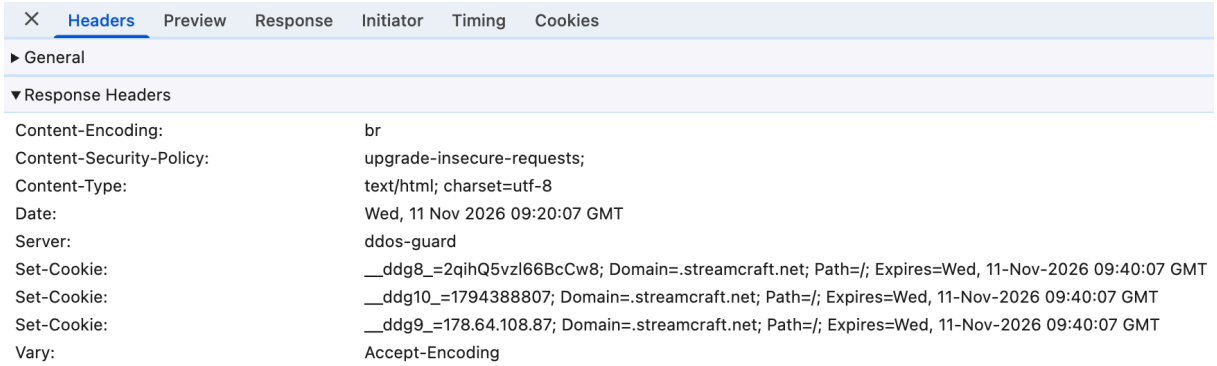


Рис. 19. Заголовки ответа

Заголовки ответа:

- `content-encoding: br` – контент передан в сжатом формате Brotli для уменьшения объема передаваемых данных и ускорения загрузки.
- `content-security-policy: upgrade-insecure-requests` – политика безопасности контента, которая предписывает браузеру автоматически перенаправлять все незащищенные (HTTP) запросы на защищенные (HTTPS), предотвращая атаки типа «человек посередине».
- `content-type: text/html; charset=utf-8` – тип контента HTML с кодировкой UTF-8.
- `date: Wed, 11 Nov 2026 09:20:07 GMT` – точное время формирования ответа сервером.

- `server: ddos-guard` – информация о том, что запрос обработан сервисом защиты от DDoS-атак (DDoS-Guard), который скрывает реальный веб-сервер.
- `set-cookie: __ddg8_=2qihQ5vzl66BcCw8; ... Expires=Wed, 11-Nov-2026 09:40:07 GMT` – установка cookie-файла `__ddg8_` для домена `.streamcraft.net` со сроком действия 20 минут. Используется системой DDoS-защиты для идентификации клиента и проверки, что запросы отправляет реальный пользователь, а не бот.
- `set-cookie: __ddg10_=1794388807; ... Expires=Wed, 11-Nov-2026 09:40:07 GMT` – установка cookie-файла `__ddg10_` (также на 20 минут) для домена `.streamcraft.net`. Служит для защиты от автоматизированных атак и ботов.
- `set-cookie: __ddg9_=178.64.108.87; ... Expires=Wed, 11-Nov-2026 09:40:07 GMT` – установка cookie-файла `__ddg9_` (на 20 минут), который, судя по значению, содержит IP-адрес клиента (178.64.108.87). Используется сервисом DDoS-Guard для проверки легитимности трафика.
- `vary: Accept-Encoding` – указывает кэширующим серверам, что ответ может различаться в зависимости от заголовка `Accept-Encoding` (т.е. от того, поддерживает ли клиент сжатие данных), чтобы не отдавать сжатый контент клиентам, которые его не поддерживают.

Рассмотрим заголовки запросов.

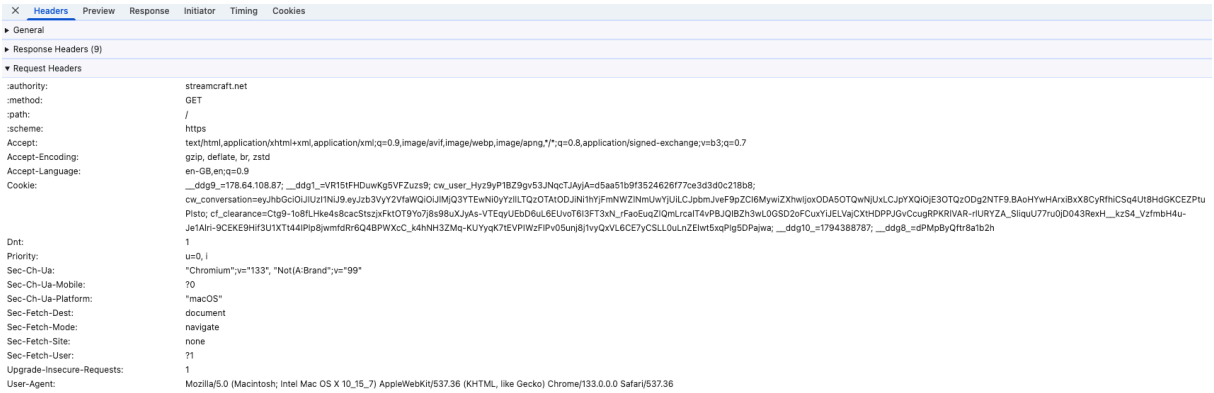


Рис. 20. Заголовки запросов

Заголовки запроса:

Псевдозаголовки HTTP/2 (Маршрутизация)

- `:authority: streamcraft.net` – доменное имя сервера, к которому отправляется запрос (аналог `Host`).
- `:method: GET` – метод запроса; означает, что клиент запрашивает данные (загрузку страницы).

- `:path: /` – путь запрашиваемого ресурса (в данном случае — корневая страница сайта).
- `:scheme: https` – используемый протокол (HTTPS), подтверждающий шифрование соединения.

Предпочтения контента (Что хочет получить браузер)

- `accept: text/html,application/xhtml+xml,...` – сообщает, какие типы контента клиент готов принять. Приоритет отдается HTML-документам и изображениям.
- `accept-encoding: gzip, deflate, br, zstd` – список поддерживаемых алгоритмов сжатия. Сервер может выбрать любой из них (например, `br` — Brotli) для уменьшения размера данных.
- `accept-language: en-GB,en;q=0.9` – предпочтительный язык контента (британский английский, но готов принять и любой другой английский).

Состояние и идентификация (Cookies и трекинг)

- `cookie: __ddg9_=...; __ddg1_=...; cw_user_...; ...` – набор cookie-файлов. Здесь видны маркеры DDoS-защиты (`__ddg9_`, `__ddg1_`, `__ddg10_`, `__ddg8_`), данные пользовательской сессии (`cw_user_...`) и сторонние трекеры (например, `_fbp` от Facebook). Это позволяет серверу идентифицировать клиента и проверить, не бот ли он.
- `dnt: 1` – заголовок «Do Not Track». Клиент просит сервер не отслеживать его действия (хотя многие сайты это игнорируют).

Приоритеты и безопасность

- `priority: u=0, i` – заголовок приоритизации запросов. Указывает, что данный запрос имеет наивысший приоритет (`u=0`) и является инкрементным (`i`).
- `upgrade-insecure-requests: 1` – просит сервер автоматически перенаправлять все незащищенные (HTTP) запросы на защищенные (HTTPS).

Client Hints и Fetch Metadata (Информация о клиенте)

- `sec-ch-ua: "Chromium";v="133", "Not(A:Brand";v="99"` – сообщает информацию о браузере (на базе Chromium версии 133).
- `sec-ch-ua-mobile: ?0` – указывает, что устройство не является мобильным (`?0` — false).
- `sec-ch-ua-platform: "macOS"` – сообщает операционную систему клиента (macOS).
- `sec-fetch-dest: document` – указывает, что запрос предназначен для загрузки документа (HTML-страницы).

- `sec-fetch-mode: navigate` – указывает, что это навигационный запрос (переход на новую страницу).
- `sec-fetch-site: none` – означает, что запрос инициирован пользователем напрямую (например, вводом адреса в строку браузера), а не со стороннего сайта.
- `sec-fetch-user: ?1` – указывает, что навигация инициирована действием пользователя (кликом по ссылке или вводом адреса).

Идентификация браузера (User-Agent)

- `user-agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36` – классическая строка, идентифицирующая браузер и ОС. В данном случае это Chrome версии 133 на macOS (упоминание Safari и Mozilla оставлено для исторической совместимости с сайтами).

Теперь рассмотрим POST-запрос. Введём тестовые данные в форму входа: `malyshevlogin:malyshevpassword`.

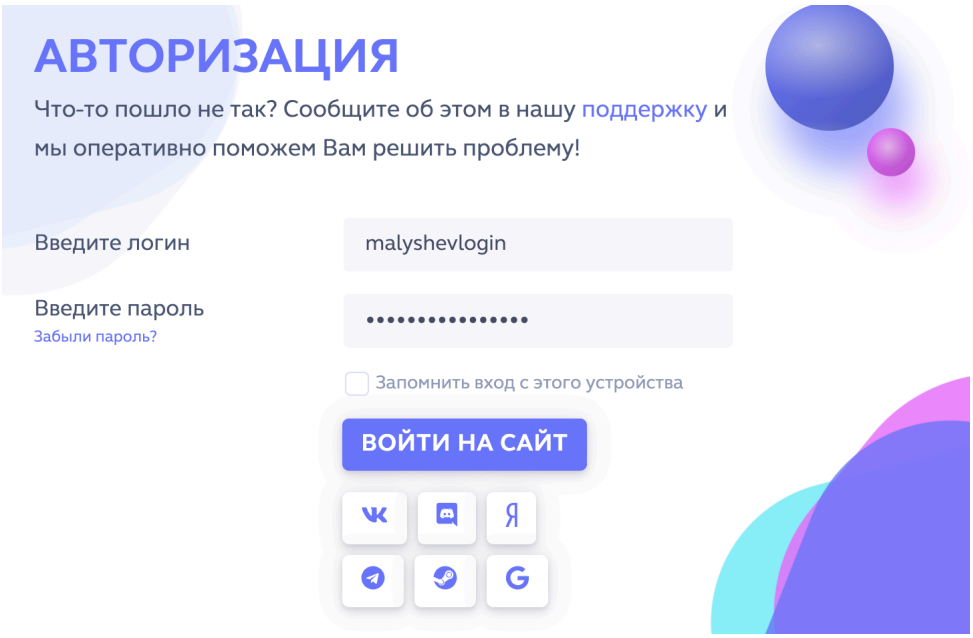


Рис. 21. Ввод тестовых данных

После ввода данных в форму появился такой POST-запрос.

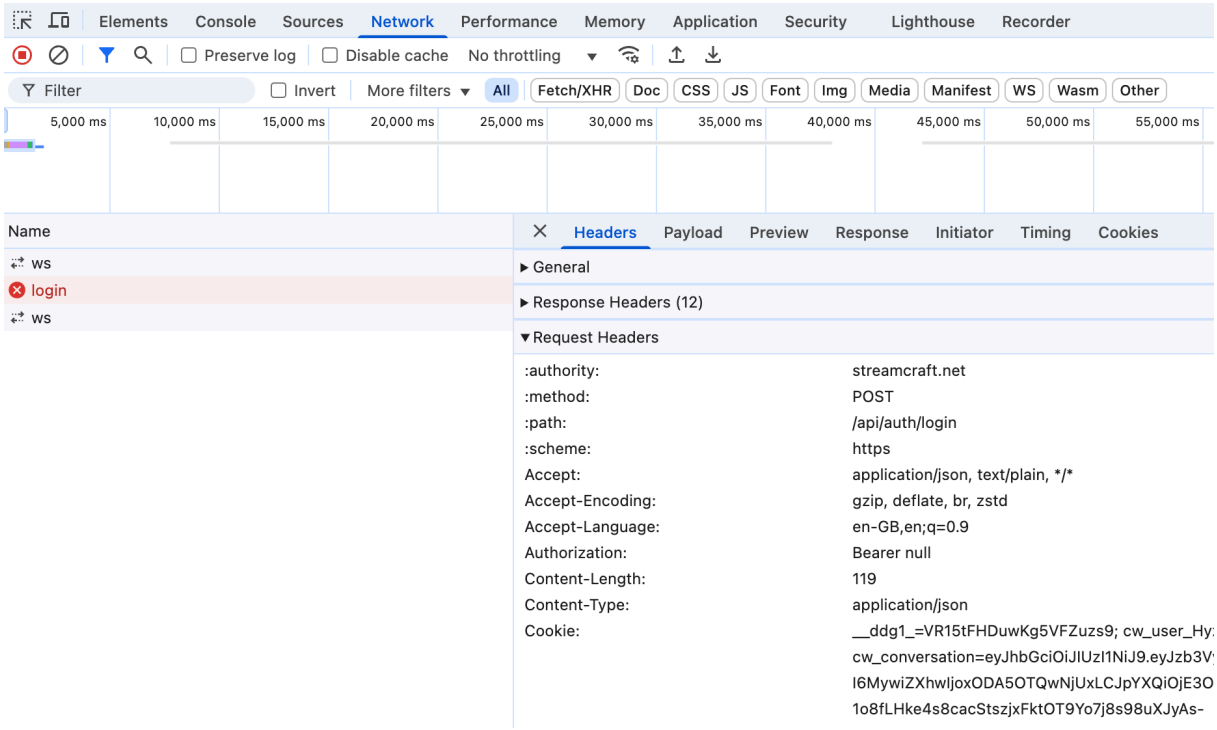


Рис. 22. POST-запрос после отправки данных формы

Видим те же общие заголовки с тем же назначением.

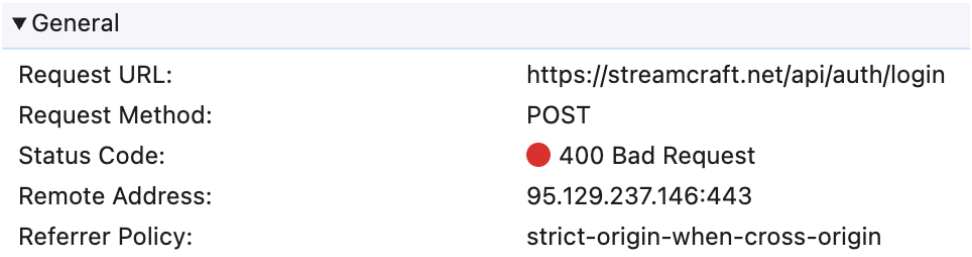


Рис. 23. Общие заголовки

Рассмотрим заголовки ответа при POST-запросе.

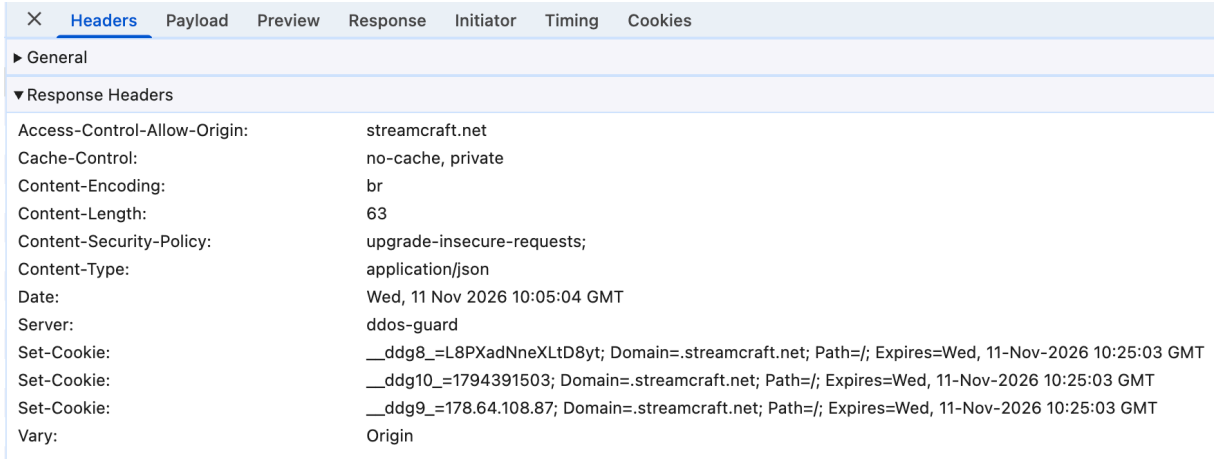


Рис. 24. Заголовки ответа при POST-запросе

Безопасность и политики

- `access-control-allow-origin: streamcraft.net` – заголовок CORS (Cross-Origin Resource Sharing). Разрешает клиентским скриптам с домена `streamcraft.net` получать доступ к этому ответу (обычно используется для API-запросов через AJAX/fetch).
- `content-security-policy: upgrade-insecure-requests` – политика безопасности контента, которая предписывает браузеру автоматически перенаправлять все незащищенные (HTTP) запросы на защищенные (HTTPS).

Информация о контенте и кэшировании

- `content-type: application/json` – тип контента. Сервер возвращает данные в формате JSON (а не HTML-страницу, как при обычном GET-запросе).
- `content-encoding: br` – контент передан в сжатом формате Brotli для уменьшения объема передаваемых данных.
- `content-length: 63` – точный размер тела ответа в байтах (после сжатия).
- `cache-control: no-cache, private` – строгая политика кэширования. Ответ является приватным (предназначен только для текущего пользователя) и требует обязательной проверки актуальности на сервере перед каждым использованием (`no-cache`), не сохраняясь в общих кэшах.

Метаданные сервера

- `date: Wed, 11 Nov 2026 10:05:04 GMT` – точное время формирования ответа сервером.
- `server: ddos-guard` – информация о том, что запрос обработан сервисом защиты от DDoS-атак (DDoS-Guard), который скрывает реальный веб-сервер.
- `vary: Origin` – указывает кэширующим серверам, что ответ может различаться в зависимости от заголовка `Origin` (заголовка, который браузер отправляет при CORS-запросах). Это гарантирует, что кэш не отдаст данные, предназначенные для одного сайта, другому.

Управление состоянием (Cookies)

- `set-cookie: __ddg8_=L8PXadNneXLtD8yt; ... Expires=Wed, 11-Nov-2026 10:25:03 GMT` – установка cookie-файла `__ddg8_` для домена `.streamcraft.net` со сроком действия 20 минут. Используется системой DDoS-защиты для идентификации клиента.
- `set-cookie: __ddg10_=1794391503; ... Expires=Wed, 11-Nov-2026 10:25:03 GMT` – установка cookie-файла `__ddg10_` (на 20 минут) для домена `.streamcraft.net`. Служит для защиты от автоматизированных атак и ботов.

- `set-cookie: __ddg9_=178.64.108.87; ... Expires=Wed, 11-Nov-2026 10:25:03 GMT` — установка cookie-файла `__ddg9_` (на 20 минут), содержащего IP-адрес клиента. Используется сервисом DDoS-Guard для проверки легитимности трафика.

Рассмотрим заголовки запроса при POST-запросе.



▼ Request Headers

:authority:	streamcraft.net
:method:	POST
:path:	/api/auth/login
:scheme:	https
Accept:	application/json, text/plain, */*
Accept-Encoding:	gzip, deflate, br, zstd
Accept-Language:	en-GB,en;q=0.9
Authorization:	Bearer null
Content-Length:	119
Content-Type:	application/json
Cookie:	__ddg1_=VR15tFHDuwKg5VFZuzs9; cw_user_Hyz9yP1BZ9gv53JNqcTJAyjA=d5aa51b9f3524626f77ce3d3d0c218b8; cw_conversation=eyJhbGciOiJIUzI1NiJ9.eyJzb3VyY2VfaWQiOiJlMjQ3YTEwNi0yYzllLTQzOTAtODJlNi1hYjFmNWZlNmUwYjUiLCJpbmJveF9pZCI6MywiZXhwIjoxODA5OTQwNjUxLCJpYXQiOjE3OTQzODg2NTF9.BAoHYwHArxiBxX8CyRfhiCSq4Ut8HdGKCEZPtuPlsto; cf_clearance=Ctg9-1o8fLHke4s8cacStszjxFktOT9Yo7j8s98uXJyAs-VTEqyUEbD6uL6EUvoT6l3FT3xN_rFaoEuqZlQmLrcalT4vPBJQlBZh3wL0GSD2oFCuxYiJELVajCXtHDPPJGvCcugRPKRlVAR-rlURYZA_SliquU77ru0jD043RexH__kzS4_VzfmbH4u-Je1Alri-9CEKE9Hif3U1XTt44lPlp8jwmfdRr6Q4BPWXcC_k4hNH3ZMq-KUYyqK7tEVPlWzFlPv05unj8j1vyQxVL6CE7yCSLL0uLnZElwt5xqPlg5DPajwa; __ddg9_=178.64.108.87; __ddg8_=pkxhZTypwLMZG0Oh; __ddg10_=1794391489
Dnt:	1
Origin:	https://streamcraft.net
Priority:	u=1, i
Referer:	https://streamcraft.net/
Sec-Ch-Ua:	"Chromium";v="133", "Not(A:Brand";v="99"
Sec-Ch-Ua-Mobile:	?0
Sec-Ch-Ua-Platform:	"macOS"
Sec-Fetch-Dest:	empty
Sec-Fetch-Mode:	cors
Sec-Fetch-Site:	same-origin
User-Agent:	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36

Рис. 25. Заголовки запроса при POST-запросе

Псевдозаголовки HTTP/2 (Маршрутизация)

- `:authority: streamcraft.net` – доменное имя сервера.
- `:method: POST` – метод запроса. Указывает, что клиент отправляет данные на сервер (в данном случае — данные для входа).
- `:path: /api/auth/login` – путь к конкретному API-методу, отвечающему за авторизацию.
- `:scheme: https` – используемый протокол (HTTPS).

Данные и авторизация (Тело запроса)

- `authorization: Bearer null` – заголовок авторизации. Клиент пытается передать токен доступа (Bearer token), но его значение — `null` (токен отсутствует). Это логично для первичного запроса на вход.
- `content-type: application/json` – указывает, что данные в теле запроса передаются в формате JSON.
- `content-length: 119` – размер тела запроса в байтах (JSON-объект с логином и паролем).

Состояние и контекст (Cookies и CORS)

- `cookie: __ddg1_=...; __ddg9_=...; ...` – набор cookie-файлов (маркеры DDoS-защиты и сессионные данные), которые браузер прикрепляет к запросу.

- `origin: https://streamcraft.net` – указывает источник запроса (сайт, с которого он был инициирован). Критически важен для CORS-проверок на сервере.
- `referer: https://streamcraft.net/` – URL страницы, с которой был сделан запрос. В данном случае — корневая страница сайта.

Fetch Metadata и Client Hints (Контекст и безопасность)

- `sec-fetch-dest: empty` – указывает, что запрос не предназначен для загрузки документа или изображения (обычно используется для fetch/XHR запросов к API).
- `sec-fetch-mode: cors` – означает, что это кросс-доменный запрос (Cross-Origin Resource Sharing).
- `sec-fetch-site: same-origin` – указывает, что запрос отправляется с того же домена, на который он направлен (streamcraft.net).
- `sec-ch-ua: "Chromium";v="133", "Not(A:Brand";v="99"` – информация о браузере (Chromium 133).
- `sec-ch-ua-mobile: ?0` – устройство не является мобильным.
- `sec-ch-ua-platform: "macOS"` – операционная система клиента.

Предпочтения и приоритеты

- `accept: application/json, text/plain, */*` – клиент ожидает получить в ответ JSON или текст (а не HTML, как при обычном переходе по ссылкам).
- `accept-encoding: gzip, deflate, br, zstd` – поддерживаемые алгоритмы сжатия.
- `accept-language: en-GB,en;q=0.9` – языковые предпочтения.
- `priority: u=1, i` – приоритет запроса. Здесь указан уровень `u=1` (обычный/средний приоритет), в отличие от навигационных запросов, где был `u=0`.
- `dnt: 1` – запрос «Do Not Track».

Идентификация браузера (User-Agent)

- `user-agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36` – строка идентификации браузера (Chrome 133 на macOS).

Если открыть полезную нагрузку, то увидим следующее.

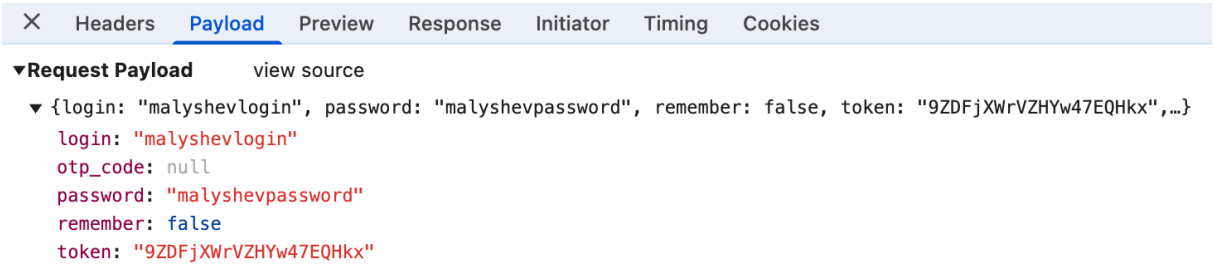


Рис. 26. Полезная нагрузка POST-запроса

Результаты анализа POST-запроса при авторизации:

Статус ответа: 400 Bad Request – указывает на то, что сервер отклонил запрос из-за ошибки на стороне клиента (некорректный формат данных, отсутствие обязательных полей или неверные параметры, в нашем случае: неверный логин или пароль).

Пароль передается в открытом виде в поле `password` со значением `malyshevpassword`. Это представляет серьезную угрозу безопасности, даже при использовании HTTPS. Хотя данные передаются по защищенному каналу, они могут быть видны в логах промежуточных прокси-серверов (которые терминируют SSL), а также сохраняться в истории форм или в инструментах разработчика (DevTools). При компрометации SSL-сертификата или самого клиентского устройства пароль становится доступен.

Проанализируем передаваемые параметры (Request Payload):

Учетные данные:

- `login: "malyshevlogin"` — логин пользователя.
- `password: "malyshevpassword"` — пароль в открытом виде (без предварительного хеширования на клиенте).
- `remember: false` — флаг «запомнить меня» (в данном случае отключен).
- `otp_code: null` — поле для одноразового пароля пустое (2FA не включена).

Технические параметры и защита:

- `token: "9ZDFjXWrVZHYw47EQHkx"` — токен аутентификации (CSRF-токен), который извлекается со страницы и передается в теле запроса для подтверждения легитимности действия. Важно отметить: в заголовках запроса `Authorization: Bearer null`, то есть сам запрос не авторизован через заголовок, а использует этот токен в теле.
- Content-Type: application/json — в отличие от классических HTML-форм, данные передаются в формате JSON, что типично для современных SPA-приложений (React, Vue и т.д.).

Таким образом, система использует API-подход с передачей JSON. Хотя применяется HTTPS-шифрование, CSRF-токен и защита от DDoS (DDoS-Guard), передача пароля в открытом виде (в теле JSON) нарушает современные стандарты безопасности. Для повышения уровня защиты рекомендуется внедрить предварительное хеширование пароля на клиенте, чтобы даже при перехвате трафика злоумышленник не получил исходный пароль.

3. Как чаще всего передаются данные аутентификации (логин/пароль) в современных веб-приложениях?

В современных веб-приложениях данные аутентификации обычно передаются исключительно через защищенное HTTPS-соединение с использованием POST-запросов и Content-Type: application/x-www-form-urlencoded или application/json, где информация помещается в тело запроса, а не в URL.

Пароли не передаются в открытом виде — вместо этого применяется предварительное хеширование на стороне клиента с использованием алгоритмов вроде bcrypt или PBKDF2 с уникальной солью. После успешной аутентификации дальнейшая работа происходит через одноразовые токены доступа (access tokens) и обновляемые refresh-токены, что исключает постоянную передачу пароля.

Обязательно используются CSRF-токены для защиты от межсайтовой подделки запросов, а сессионные данные хранятся в HTTP-only cookies с настройками SameSite, недоступные из JavaScript. Дополнительную безопасность обеспечивают современные заголовки HTTP Strict Transport Security, CSP и X-Content-Type-Options.

В передовых реализациях все чаще применяются протоколы OAuth 2.0, OpenID Connect, а также беспарольные методы аутентификации через email-ссылки, биометрию и многофакторную аутентификацию, что в совокупности обеспечивает надежную защиту учетных данных пользователей.

4. Вывод

В ходе лабораторной работы я разобрался, как браузер управляет безопасностью и как выглядит сетевой трафик изнутри. В настройках Chrome я

по максимуму ограничил отслеживание: заблокировал сторонние cookie, отключил темы рекламы и оценку её эффективности, включил усиленную защиту и безопасный DNS. Также я запретил сайтам доступ к микрофону, камере и железу, оставив только JavaScript и картинки, без которых сайты просто не работают.

Самое интересное началось при анализе трафика на сайте streamcraft.net. Я посмотрел, из чего состоят GET и POST запросы, и понял, что даже при включенном HTTPS пароль передается в открытом виде прямо в теле JSON-запроса. Это стало для меня главным открытием: защищенное соединение не спасает, если внутри передаются чистые данные. В итоге я сделал вывод, что современные приложения должны хешировать пароль на клиенте и использовать токены, а не гонять секреты в открытом виде, пусть и по защищенному каналу.